ST-ROBOT · TÀI LIỆU TỰ HỌC

LEGO SPIKE Prime

Từ cơ cấu truyền động đến mô hình phân loại màu có kiểm chứng.

6 bài học · Lý thuyết, ví dụ, thực hành, kiểm tra và tài liệu tham khảo

Lộ trình học tập

1. Cơ cấu và góc quay
2. Đọc và phân biệt màu
3. Phân loại không đếm lặp
4. Dự án: trạm phân loại
5. Bánh răng: tốc độ và mô-men
6. Máy trạng thái cho bộ phân loại

Cách sử dụng tài liệu

Đọc mục tiêu và kiến thức nền; tự dự đoán trước ví dụ; thực hành theo các bước; ghi lại dữ liệu và giải thích sai lệch. Sau khi tự làm, đối chiếu đáp án và tự kiểm tra. Phòng lab bên ngoài không tự đồng bộ điểm; hoạt động mô phỏng không thay thế kiểm tra an toàn phần cứng thật.

Bài 1/6 · Cơ cấu và góc quay

Cho mô hình tay gạt đi ra rồi trở về, so sánh góc motor với góc đầu ra và nhận biết vị trí nghỉ.

Chuẩn bị

Chưa cần viết Python hoặc tính tỷ số truyền. Nhận biết tay chắn, động cơ (motor) và vị trí nghỉ; 90° là một góc vuông. Bắt đầu bằng mô phỏng trong Lab. Nếu dùng kit thật, cần người hướng dẫn kiểm tra cơ khí.

Sau bài học, bạn có thể

1. Dự đoán rồi đối chiếu góc motor và góc đầu ra của một mô hình cơ cấu.

2. Chỉ đúng vị trí nghỉ và kiểm tra chuỗi chuyển động có trở về mốc.

3. Phân biệt mô hình bánh răng 12:36 với tay chắn trực tiếp của Cage Bot.

Cùng hiểu bài từ đầu

Khi bạn ra lệnh motor quay 90°, tay gạt có nhất thiết quay 90° không? Câu trả lời phụ thuộc những gì nằm giữa trục motor và tay gạt. Bài này bắt đầu bằng một bộ truyền hai bánh răng, đánh dấu vị trí nghỉ rồi theo dõi đầu vào và đầu ra riêng. Hiểu đường truyền chuyển động giúp bạn tránh nhầm lệnh phần mềm với góc mà cơ cấu thật đã đạt.

Đánh một dấu trên mỗi bánh răng rồi quay bánh nhỏ chậm bằng tay: răng bánh này đẩy răng bánh kia.

Bánh vào 12 răng quay một vòng đưa 12 răng qua chỗ ăn khớp. Bánh ra 36 răng chỉ đi qua 12/36 vòng, nên quay chậm hơn và ngược chiều với cặp bánh ngoài.

Giới hạn của ví von: Mô tả này giả sử các răng ăn khớp lý tưởng. Độ rơ, ma sát, liên kết khác và chặn cơ khí có thể làm góc tay gạt thực khác phép tính.

Những từ cần hiểu

1. Trục vào và trục ra: Trục vào nhận chuyển động từ motor; trục ra truyền chuyển động tới tay gạt. Phân biệt chúng trước khi gắn một con số góc.

2. Vị trí nghỉ: Mốc được chọn để đo góc, thường gọi 0°. Bật Hub không tự chứng minh cơ cấu đang ở mốc này.

3. Góc có dấu: Dấu +/− chỉ hai chiều quay theo quy ước. Với hai bánh răng ngoài, đầu ra có dấu ngược đầu vào; độ lớn là khoảng góc quay.

4. Độ rơ: Khoảng chuyển động mất khi đảo chiều do khe hở của cơ cấu. Motor bắt đầu quay ngược không bảo đảm đầu ra đã đổi ngay cùng lúc.

Đi ra và về với bộ truyền 12:36

Bắt đầu với: Mô hình lý tưởng, bánh vào 12 răng, bánh ra 36 răng; tay gạt gắn trực tiếp trục ra. Chọn vị trí nghỉ 0° và góc motor 0° → 90° → 0°.

Xác định mốc và chiều

Cả góc motor và góc đầu ra đo từ mốc đã đặt. Chọn chiều dương cho đầu vào; chiều ngược được biểu diễn bằng dấu âm.

Tính lúc đi ra

Góc đầu ra = −90° × 12/36 = −30°. Tay gạt đổi độ lớn 30°, không phải 90°.

Đưa về mốc

Khi motor trở lại 0° từ cùng mốc, mô hình lý tưởng cho đầu ra 0°. Cần quan sát đầu ra thực hoặc mô hình, không chỉ nhìn lệnh đã gửi.

Đọc kết quả trong đúng mô hình

GearsBot Cage Bot có tay chắn truyền động trực tiếp riêng; chuỗi góc tay chắn 0° → 30° → 0° ở đó không phải phép kiểm chứng bộ bánh răng SPIKE 12:36 này.

Kết quả: Bảng mô hình: motor 0°,90°,0° tương ứng đầu ra 0°,−30°,0°. Đơn vị là độ góc, ký hiệu °.

Số răng quyết định tỷ số góc lý tưởng. Trên cơ cấu thật còn phải xác định gốc, giới hạn và cách dừng khi không đạt; giá trị tính được không cho phép vượt chặn cơ khí.

Tự giải thích rồi đối chiếu

Muốn độ lớn đầu ra 20° trong cùng mô hình, motor cần đổi bao nhiêu?

60°, vì 60 × 12/36 = 20. Chiều đầu ra ngược motor; phải xét giới hạn cơ khí trước khi thử thật.

Motor đã nhận lệnh về 0° có đủ chứng minh tay gạt thực đã về nghỉ không?

Chưa. Cần quan sát hoặc phản hồi phù hợp; tải, kẹt, độ rơ hoặc mốc đặt sai đều có thể ảnh hưởng.

Chọn độ sâu phù hợp

Mới bắt đầu · Quan sát hai góc riêng

Chưa cần nhớ công thức. Thay motor từ 0° đến 90° rồi về 0°, ghi góc đầu ra và chỉ chiều quay trên sơ đồ.

Dùng hai cột “góc motor” và “góc tay gạt”; khoanh chỗ hai số khác nhau.

Đã học cơ bản · Giải thích bằng số răng

Một góc đầu vào đưa cùng số răng đi qua điểm ăn khớp ở hai bánh. Bánh ra nhiều răng hơn cần quay một phần vòng nhỏ hơn.

Tính và thử trong mô hình motor 30°,60°,90°; so sánh tỷ số góc.

Muốn tìm hiểu sâu · Từ mô hình sang cơ cấu thật

Công thức chỉ đúng cho quan hệ truyền động đã giả định. Tay gạt qua liên kết không gắn trực tiếp có thể cần hình học khác; encoder motor không đo trực tiếp mọi đầu ra.

Đề xuất phép đo góc đầu ra khi đi tới cùng một vị trí từ hai chiều để khảo sát độ rơ, với góc nhỏ trong phạm vi đã xác nhận an toàn.

Kiến thức trọng tâm

Việc đầu tiên: đi đến một góc rồi trở về

Động cơ (motor) quay trục vào; bánh răng truyền chuyển động sang trục ra gắn tay gạt. Trước hết chọn mốc nghỉ 0°, thay một góc motor và đọc góc đầu ra. Hai góc có thể khác nhau vì cơ cấu truyền động. Trong mô hình 12:36 của bài, motor 90° cho đầu ra có độ lớn 30° và ngược chiều. Sau khi thử, đọc công thức để giải thích; tay chắn trực tiếp của GearsBot Cage Bot là một hoạt động riêng.

Đọc thêm · Lệnh motor không phải góc tay gạt

Motor làm quay trục vào; bánh răng hoặc liên kết truyền chuyển động ra cơ cấu. Với một cặp bánh răng ngoài lý tưởng, góc trục ra có độ lớn $|\theta ra| = |\theta vào| \times Zvào/Zra$ và ngược chiều quay. Z là số răng. Nếu tay gạt lắp trực tiếp trục ra thì góc tay gạt bằng góc trục ra trong mô hình cứng; liên kết phức tạp cần xét hình học riêng.

Đọc thêm · Gốc tọa độ và giới hạn cơ khí

Vị trí nghỉ là mốc đo góc được xác định trước, không tự có chỉ vì vừa bật Hub. Nêu phạm vi được phép, chẳng hạn từ −30° đến +30° quanh vị trí nghỉ, nhưng phải chọn theo cơ cấu thực. Không ép motor vào chặn cứng để tìm gốc khi chưa có quy trình phù hợp; có thể gây kẹt, quá tải hoặc hỏng linh kiện.

Đọc thêm · Kiểm tra khi có tải

Độ rơ khiến đổi chiều chưa tạo ngay chuyển động đầu ra; tải và ma sát có thể làm motor không đạt góc yêu cầu. Dùng góc nhỏ, tốc độ thấp và có timeout/điều kiện dừng khi thử. GearsBot hỗ trợ luyện thuật toán motor trên mô hình riêng, không mô phỏng chính xác một bộ SPIKE tự lắp hay mọi tỷ số truyền của nó.

Ví dụ có lời giải

Bánh chủ động 12 răng kéo bánh bị động 36 răng. Motor quay 90° thì trục ra lý tưởng quay 30° ngược chiều; nếu tay gạt gắn trực tiếp trục ra, tay gạt đổi góc 30°. Đây không phải quyền cho phép chạy qua chặn cơ khí của tay gạt.

Lỗi thường gặp và cách tránh

1. Motor quay 90° không luôn khiến tay gạt quay 90°.

2. Cơ cấu lắp chắc vẫn có thể quá tải hoặc kẹt; không được dùng motor để ép qua chặn cứng.

Các bước thực hành

1. Dự đoán: trên mô hình có cặp bánh răng 12:36, khi motor quay 90° thì tay gạt đầu ra sẽ quay cùng góc hay một góc khác? Vẽ phỏng đoán và đánh dấu vị trí nghỉ 0°.

2. Thao tác: vào Lab → Bàn thí nghiệm, giữ cặp bánh răng 12:36 và đặt góc motor lần lượt 0°, 90°, 0°. Chạy mô hình ở mỗi mốc; chỉ thay góc motor.

3. Quan sát: ghi góc motor và góc đầu ra ở ba mốc. Đối chiếu xem tay gạt có về 0° không và giải thích đầu ra −30° có độ lớn 30°, ngược chiều dương đã chọn.

4. Giải thích: chỉ motor, bánh răng vào 12 răng, bánh răng ra 36 răng và tay gạt trên sơ đồ. Viết một câu: thêm bộ truyền có thể đổi góc và chiều quay.

5. Chạy thử mở rộng: chọn GearsBot Cage Bot theo hướng dẫn Lab, dán toàn bộ mẫu Python rồi Reset và Run. Quan sát chuỗi tay chắn 0° → 30° → 0° và các cặp target_deg/measured_deg. Đây là tay

chắn truyền động trực tiếp của Cage Bot, khác mô hình bánh răng 12:36.

6. Mở rộng: đọc công thức để tự tính motor 60° tạo đầu ra bao nhiêu. Nếu thử kit thật, nhờ người hướng dẫn kiểm tra cơ khí, mốc nghỉ và giới hạn; dừng khi kẹt hoặc quá hạn.

Mã mẫu · Python · GearsBot Cage Bot (ev3dev2)

```
# GearsBot Cage Bot, ev3dev2 Python. Press Reset before Run.
from ev3dev2.motor import LargeMotor, OUTPUT_C, SpeedDPS
from ev3dev2.sensor.lego import ColorSensor, UltrasonicSensor
from time import time, sleep

try:
  color = ColorSensor("in3")
  sonar = UltrasonicSensor("in4")
  arm = LargeMotor(OUTPUT_C)
except Exception as error:
  print("SETUP:", error)
  raise RuntimeError("Select Cage Bot, then press Reset.")

def move_arm(target, deadline):
  if not 0 <= target <= 45:
    raise ValueError("Use an arm target from 0 to 45 degrees.")
  if time() >= deadline:
    return False
  limit = min(deadline, time() + 3.0)
  arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
  while abs(arm.position - target) > 2:
    if time() >= limit:
      arm.off(brake=True)
      return False
    sleep(0.02)
  arm.off(brake=True)
  return time() < limit

# Cage Bot arm angles are direct encoder angles, not a 12:36 gearbox.
deadline = time() + 15.0
try:
  for target in [0, 30, 0]:
    if not move_arm(target, deadline):
      print("ERROR: target timeout", target, "measured", arm.position)
      break
    print("target_deg", target, "measured_deg", arm.position)
    sleep(0.3)
finally:
  arm.off(brake=True)

# Separate mathematical prediction for an external 12:36 gear pair.
teeth_in, teeth_out = 12, 36
motor_deg = 90.0
output_deg = -motor_deg * teeth_in / teeth_out
required_motor_deg = -20.0 * teeth_out / teeth_in
print("model_output_deg", output_deg)
print("motor_deg_for_positive_20_deg_output", required_motor_deg)
```

Chọn và sử dụng phòng lab

Bắt đầu ở Bàn thí nghiệm: giữ cặp bánh răng 12:36, dự đoán rồi thử góc motor 0°, 90°, 0° và ghi góc đầu ra. Mô hình tính quan hệ góc, không chạy Python hay mô phỏng kẹt cơ khí. Khi làm phần chương trình, chọn GearsBot → Robot → Select Robot → Cage Bot, mở Python, dán toàn bộ mẫu rồi Reset và Run. Mẫu dùng tay chắn outC, cảm biến màu in3, siêu âm in4; A/B là hai bánh xe. Quan sát 0° → 30° → 0° và target_deg/measured_deg; dừng khi báo timeout rồi kiểm tra trước khi Reset. Góc Cage Bot là góc trực tiếp, còn phép tính bánh răng cuối chương trình là mô hình riêng. Với Hub SPIKE thật, dùng ứng dụng LEGO Education SPIKE và cấu hình cổng của bộ lắp.

Bài tập áp dụng

Trên mô hình bánh răng 12:36, dự đoán rồi thử góc motor 0° → 90° → 0°; ghi góc đầu ra và giải thích vì sao có thể khác góc motor. Nêu mốc nghỉ và việc cần làm khi cơ cấu thật không về được mốc. Phần thêm: muốn đầu ra đổi độ lớn 20° thì motor cần đổi bao nhiêu độ?

Gợi ý đối chiếu sau khi tự làm

Mô hình cho đầu ra 0° → -30° → 0°. Với một cặp bánh răng ngoài 12:36, độ lớn góc đầu ra bằng một phần ba góc motor, chiều ngược lại. Vì vậy đầu ra 20° cần motor đổi độ lớn 60°. Mốc nghỉ là 0° đã chọn trước; khi cơ cấu thật không về đúng hạn thì dừng motor và kiểm tra. Chương trình Cage Bot riêng dùng chuỗi trực tiếp 0° → 30° → 0°, không có bộ truyền 12:36 này.

Tự kiểm tra mức hiểu

1. Trong mô hình 12:36, motor quay 90° thì đầu ra quay bao nhiêu độ và theo chiều nào?

2. Nếu cơ cấu chưa về mốc đúng hạn, vì sao cần kiểm tra trước khi chạy lại?

Câu hỏi trắc nghiệm

Vì sao góc tay gạt có thể khác góc động cơ?

1. Do tỷ số truyền và cơ cấu

2. Do tên chương trình

3. Do màu Hub

Đáp án: 1. Bộ truyền và liên kết cơ khí quyết định tỷ lệ góc, chiều quay và độ rơ. Tên chương trình hay màu Hub không tạo ra tỷ số truyền.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

GearsBot — cấu hình Cage Bot chính thức

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GearsBot — Python ev3dev2 motor API

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GearsBot — Python ev3dev2 cảm biến màu và siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 2/6 · Đọc và phân biệt màu

Phân biệt màu thật của vật và kết quả phân loại do cảm biến trả về.

Chuẩn bị

Biết câu điều kiện nếu–nếu không, đọc bảng kết quả và giữ cố định một biến khi so sánh.

Sau bài học, bạn có thể

1. Phân biệt màu thật của vật và kết quả phân loại do cảm biến trả về.

2. Thiết kế bảng đo có đỏ, xanh và trường hợp chưa xác định.

3. Viết nhánh điều kiện loại trừ nhau và xử lý kết quả không hợp lệ an toàn.

Cùng hiểu bài từ đầu

Màu bạn nhìn thấy trên vật và nhãn cảm biến trả về là hai thứ liên quan nhưng không đồng nhất. Cảm biến nhận ánh sáng phản xạ trong một điều kiện đặt cụ thể; phần mềm dùng số đo đó để chọn nhãn. Vì vậy một vật đỏ có thể có lượt đọc chưa xác định. Điều cần học là ghi đúng kết quả đo và xử lý sự không chắc chắn, thay vì ép mọi đầu vào thành đỏ hoặc xanh.

Hãy hình dung nhận màu một chiếc áo qua ảnh: ánh sáng và góc chụp có thể làm cùng chiếc áo trông khác nhau.

Cảm biến cũng phụ thuộc ánh sáng phản xạ, khoảng đặt và bề mặt. Nhãn đỏ/xanh là kết quả diễn giải số đo, không phải bản thân vật.

Giới hạn của ví von: Cảm biến màu có nguồn sáng, cách thu và thuật toán riêng, khác mắt người hay máy ảnh. Ví dụ chỉ giúp hiểu ảnh hưởng điều kiện đo, không dự đoán chính xác độ sai của từng thiết bị.

Những từ cần hiểu

1. Nhãn màu: Tên như đỏ, xanh hoặc chưa xác định do hệ thống phân loại trả về. Đây là dữ liệu phân loại, không phải số đo độ phản xạ.

2. Độ phản xạ: Đại lượng mô tả mức ánh sáng thu về theo thang của cảm biến. Hai vật khác màu vẫn có thể có mức phản xạ gần nhau.

3. Chưa xác định: Hệ thống chưa gán được nhãn đáng tin trong tập đang dùng. Có thể do vật lạ, ngoài tầm hoặc điều kiện đo, không tự chứng minh vùng trống.

4. Nhánh loại trừ nhau: Mỗi lượt chọn đúng một trong các nhánh đỏ, xanh, còn lại; if/elif/else biểu diễn lựa chọn này rõ hơn những if rời dễ gắn nhầm else.

Một vật, năm lượt đọc

Bắt đầu với: Giữ cùng một vật đỏ ở vị trí đã chọn. Nhãn lần lượt: đỏ, đỏ, chưa xác định, đỏ, đỏ.

Ghi sự thật quan sát

Ghi nguyên năm nhãn theo thứ tự, giữ nguyên nhãn chưa xác định ở lượt 3. Không sửa dữ liệu theo màu mình biết trước.

Đếm nhãn phù hợp

Có 4 lượt nhận đỏ trên 5 lượt, tức 4/5 = 80% lượt đọc trong điều kiện này. Có 1 lượt chưa xác định, không phải một vật mới.

Chọn nhánh

Bốn lượt đỏ đi nhánh A; lượt chưa xác định đi nhánh kiểm tra/chưa phân loại. Không tự đổi lượt đó thành xanh hay đỏ.

Tìm yếu tố ảnh hưởng

Đo một nhóm mới khi chỉ thay khoảng đặt và giữ các yếu tố còn lại. So sánh bảng để có bằng chứng thay vì cùng lúc đổi cả ánh sáng lẫn góc.

Kết quả: Bảng có năm mẫu nhưng chỉ một vật. Kết quả mô tả khả năng đọc trong điều kiện đã ghi, chưa phải tỷ lệ phân loại toàn trạm.

Tách vật thật, mẫu đo và quyết định giúp tránh suy luận quá mức. Trên bàn logic, màu do người dùng chọn chỉ kiểm tra nhánh; nó không mô phỏng phép đo quang học của kit thật.

Tự giải thích rồi đối chiếu

Một lượt chưa xác định có đủ kết luận không có vật không?

Không. Vật có thể còn đó nhưng ngoài tầm hoặc đọc không ổn định; cần tín hiệu hiện diện phù hợp nếu muốn xác nhận trống.

Đọc đúng màu có đảm bảo vật tới đúng ngăn không?

Chưa. Cơ cấu gạt và điều khiển phía sau vẫn có thể lỗi; đó là kết quả của toàn quy trình.

Chọn độ sâu phù hợp

Mới bắt đầu · Tách điều biết và điều đo

Ghi hai cột: màu vật bạn chuẩn bị và nhãn cảm biến báo. Khi hai cột khác nhau, giữ cả hai để hiểu lỗi.

Với đỏ, xanh, chưa xác định, hãy chọn nhánh A, B, kiểm tra mà không cần motor.

Đã học cơ bản · Đo lặp có kiểm soát

Giữ vật, khoảng cách, góc và ánh sáng ổn định trong một nhóm đo. Sau đó đổi một yếu tố để xem sự khác nhau gắn với yếu tố nào.

Thiết kế hai nhóm năm lượt chỉ khác khoảng đặt; ghi những điều phải giữ nguyên.

Muốn tìm hiểu sâu · Đánh giá từng nhãn

Một tỷ lệ chung có thể che trường hợp xanh thường bị nhận thành đỏ. Bảng màu kỳ vọng so với nhãn đo cho biết lỗi nhầm cụ thể; chưa xác định cần cột riêng.

Lập bảng đếm cho hai vật đỏ/xanh với ba đầu ra đỏ/xanh/chưa biết; nêu cách dùng bảng để chọn thử nghiệm tiếp.

Kiến thức trọng tâm

Cảm biến đo ánh sáng rồi phần mềm phân loại

Cảm biến màu chiếu/thu ánh sáng phản xạ để tạo số đo, từ đó ước lượng nhãn màu hoặc độ phản xạ. Nhãn màu và giá trị phản xạ là hai loại dữ liệu khác nhau. Khoảng cách, góc đặt, ánh sáng môi trường và bề mặt bóng/mờ có thể làm đổi kết quả dù vật vẫn như cũ.

Phân biệt chưa biết và không có vật

Một nhãn không xác định có thể do không có vật, vật ngoài tầm, vật màu lạ hoặc đọc không ổn định. Không được suy mọi nhãn lạ là đỏ. Nếu không có cảm biến hiện diện riêng, phải thừa nhận giới hạn suy luận từ màu; một tín hiệu "không xác định" không luôn đủ chứng minh vật đã được lấy ra.

Nhánh quyết định cần loại trừ nhau

Dùng cấu trúc nếu đỏ → nhóm A; nếu không mà xanh → nhóm B; còn lại → vùng kiểm tra/chưa phân loại. Chuỗi hai câu nếu rời nhau có thể khiến else gắn với câu nếu thứ hai, tạo hành vi khác ý định. Kiểm thử cả nhãn không biết và ghi kết quả thay vì cưỡng bức chọn một nhóm.

Ví dụ có lời giải

Giữ cố định một vật đỏ và khoảng cách đã chọn, đo 5 lượt. Nếu kết quả là đỏ, đỏ, chưa biết, đỏ, đỏ thì ghi 4/5 lượt nhận đỏ, một lượt chưa xác định. Không đổi lượt lỗi thành đỏ để làm đẹp bảng; thử thay ánh sáng hoặc khoảng cách từng yếu tố một.

Lỗi thường gặp và cách tránh

1. Nhãn màu không phải bản chất tuyệt đối của vật, và độ phản xạ không phải nhãn màu.

2. Năm lần đọc không có nghĩa năm vật; một vật đứng yên có thể tạo ra nhiều mẫu.

Các bước thực hành

1. Lập bảng gồm vật kỳ vọng, khoảng cách/góc đặt, ánh sáng, nhãn đo được. Thử lần lượt vật đỏ, xanh và không đặt vật.

2. Trong bàn logic, chọn từng màu rồi đọc để kiểm tra ba nhánh; nhớ đây là đầu vào do người dùng chọn, không phải số đo cảm biến quang học.

3. Nếu có kit: giữ một điều kiện đặt vật cố định, đọc ít nhất 5 lượt mỗi loại; tiếp theo chỉ thay khoảng cách rồi đo lại.

4. Ghi riêng nhãn chưa xác định. Dùng cấu trúc if/elif/else hoặc khối điều kiện tương ứng để nhánh đỏ/xanh/khác không chồng nhau.

Mã mẫu · Python · GearsBot Cage Bot (ev3dev2)

```
# GearsBot Cage Bot, ev3dev2 Python. Press Reset before Run.
from ev3dev2.motor import LargeMotor, OUTPUT_C, SpeedDPS
from ev3dev2.sensor.lego import ColorSensor, UltrasonicSensor
from time import time, sleep

try:
  color = ColorSensor("in3")
  sonar = UltrasonicSensor("in4")
  arm = LargeMotor(OUTPUT_C)
except Exception as error:
  print("SETUP:", error)
  raise RuntimeError("Select Cage Bot, then press Reset.")

def move_arm(target, deadline):
  if not 0 <= target <= 45:
    raise ValueError("Use an arm target from 0 to 45 degrees.")
  if time() >= deadline:
    return False
  limit = min(deadline, time() + 3.0)
  arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
  while abs(arm.position - target) > 2:
    if time() >= limit:
      arm.off(brake=True)
      return False
    sleep(0.02)
  arm.off(brake=True)
  return time() < limit

deadline = time() + 60.0
last = None
try:
  if not move_arm(45, deadline):
    raise RuntimeError("Arm opening timeout; inspect and Reset.")
  print("Ready: approach a colored object with the joystick.")
  while time() < deadline:
    distance = sonar.distance_centimeters
    raw = color.color
    if raw == 5:
      group = "A"
    elif raw == 2:
      group = "B"
    else:
      group = "CHECK"
    near = 0 < distance <= 3.0
    reading = (raw, group, near)
    if reading != last:
      print("color_id", raw, "group", group,
          "distance_cm", round(distance, 1), "near", near)
      last = reading
    sleep(0.1)
finally:
  arm.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn Robot → Select Robot → Cage Bot, chuyển sang Python rồi bấm Reset trước mỗi lần chạy. Mẫu dùng ev3dev2 của GearsBot: outC là tay chắn, in3 là cảm biến màu hướng trước, in4 là siêu âm; cổng A/B dành cho hai bánh xe. Chuẩn bị vật đỏ, xanh và vàng trên sân qua Worlds → World Builder; đặt mặt màu ngang tầm cảm biến. Mở joystick để điều khiển bánh xe: tiếp cận chậm đến khoảng cách ≤ 3 cm, dừng bánh, sau chu kỳ lùi ra ≥ 8 cm. Mốc 3 cm phù hợp tầm đọc 5 cm của cảm biến màu Cage Bot. Mã mở tay chắn lên 45°, theo dõi cảm biến trong 60 giây và in mỗi khi nhãn hoặc trạng thái ở gần thay đổi: đỏ → A, xanh → B, còn lại → CHECK. Đọc nhãn màu và khoảng cách ở hai cột riêng; CHECK giữ nghĩa cần kiểm tra, còn hiện diện do siêu âm xác định. Đây là mã chạy cho mô hình Cage Bot; với Hub SPIKE thật, dùng API của ứng dụng LEGO Education SPIKE và cấu hình cổng của bộ lắp.

Bài tập áp dụng

Cho các nhãn đo đỏ, xanh, chưa biết, đỏ, chưa biết. Lập bảng nhánh xử lý cho từng lượt. Nếu cả 5 lượt đều là một vật đỏ đứng yên, em có được kết luận có hai vật lạ không? Vì sao?

Gợi ý đối chiếu sau khi tự làm

Các nhánh là A, B, chưa phân loại, A, chưa phân loại. Không thể đếm số vật chỉ từ số mẫu màu; nhiều mẫu có thể thuộc cùng một vật và kết quả sai/không rõ. Cần thông tin hiện diện và một chu kỳ vật vào–vật ra để đếm.

Tự kiểm tra mức hiểu

1. Phân biệt được nhãn màu và cường độ phản xạ.

2. Viết được ba nhánh loại trừ đỏ/xanh/khác.

3. Ghi được ít nhất một nguyên nhân nhãn chưa xác định mà không kết luận ngay là không có vật.

Câu hỏi trắc nghiệm

Khi màu chưa xác định, nên...

1. Tự coi là đỏ

2. Giữ nhánh chưa phân loại

3. Quay motor liên tục

Đáp án: 2. Nhánh chưa phân loại giữ rõ sự không chắc chắn khi cảm biến không cho một nhãn hợp lệ. Tự coi là đỏ sẽ làm mất thông tin lỗi và có thể gạt vật sai nhóm.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

GearsBot — cấu hình Cage Bot chính thức

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GearsBot — Python ev3dev2 motor API

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GearsBot — Python ev3dev2 cảm biến màu và siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 3/6 · Phân loại không đếm lặp

Giải thích vì sao đọc cảm biến nhiều lần có thể gây đếm/gạt lặp cho một vật.

Chuẩn bị

Hoàn thành bài đọc màu và hiểu nhãn chưa xác định. Biết biến Boolean đúng/sai và câu điều kiện kết hợp bằng VÀ.

Sau bài học, bạn có thể

1. Giải thích vì sao đọc cảm biến nhiều lần có thể gây đếm/gạt lặp cho một vật.

2. Thiết kế cờ đã xử lý chỉ được đặt lại khi vật rời vùng cảm biến.

3. Tách số chu kỳ nhận vật khỏi số chu kỳ phân loại thành công.

Cùng hiểu bài từ đầu

Robot đọc cảm biến nhiều lần trong lúc một vật còn đứng trước nó. Nếu cứ thấy đỏ là gạt và tăng đếm, một vật sẽ bị xử lý nhiều lần. Ta cần thêm trí nhớ: vật hiện tại đã được xử lý chưa? Bài này theo dõi mẫu đo, sự xuất hiện của vật mới và hoàn tất chu kỳ như những sự kiện riêng. Nhờ vậy số đếm có ý nghĩa rõ ràng.

Một người kiểm vé cần đánh dấu vé đã kiểm; nhìn lại cùng chiếc vé không được tính là một người mới.

Cờ đã_xử_lý giữ dấu cho vật hiện tại. Chỉ khi xác nhận vật cũ rời vùng nhận, hệ thống mới cho phép xử lý vật kế tiếp.

Giới hạn của ví von: Một nhãn màu chưa biết không giống một chiếc vé đã rời bàn. Cần bằng chứng trống đáng tin; chờ một khoảng thời gian cố định cũng không bảo đảm vật đã đi.

Những từ cần hiểu

1. Mẫu đo: Một lần lấy dữ liệu cảm biến. Một vật có thể tạo nhiều mẫu, ví dụ 20 mẫu nếu đọc 20 lần/s trong 1 s.

2. Vật mới: Một sự kiện được xác định theo quy tắc vào–ra/hiện diện. Nó không đồng nghĩa với mọi mẫu có nhãn màu.

3. Cờ đã xử lý: Biến đúng/sai nhớ rằng vật hiện tại đã được chốt xử lý. Cờ chặn khởi động lại chu kỳ khi còn cùng vật.

4. Bộ đếm thành công: Chỉ tăng khi chu kỳ hoàn tất theo tiêu chí đã đặt. Khác với số vật nhận vào hoặc số lần đã khởi động gạt.

Một vật đỏ rồi một vật xanh

Bắt đầu với: Ban đầu vùng trống, cờ sai, số vật nhận vào = 0 và thành công = 0. Mẫu hiện diện/màu: đỏ, đỏ, đỏ, trống, xanh, xanh, trống; giả sử mỗi chu kỳ đều hoàn tất.

Đỏ đầu tiên

Có vật hợp lệ và cờ sai nên chốt một chu kỳ, đặt cờ đúng. Số nhận vào tăng 1; chỉ sau khi chu kỳ hoàn tất mới tăng thành công lên 1.

Hai mẫu đỏ tiếp

Cờ vẫn đúng nên không mở chu kỳ khác. Dữ liệu được đọc lại nhưng vẫn thuộc vật đang có.

Trống rồi xanh

Khi xác nhận trống, cho phép vật kế tiếp. Xanh đầu tiên mở chu kỳ thứ hai; xanh lặp không mở thêm. Trống cuối cho phép bắt đầu chu kỳ sau.

Kết quả: Có 5 mẫu màu nhưng 2 vật mới và 2 chu kỳ thành công. Nếu chu kỳ xanh bị kẹt thì nhận vào vẫn là 2, thành công chỉ là 1.

Cờ cung cấp bộ nhớ giữa các lượt đọc. Tách chốt chống lặp khỏi báo thành công giúp lỗi cơ khí không biến thành một lượt phân loại đúng hoặc gây khởi động gạt lại liên tục.

Tự giải thích rồi đối chiếu

Đang xử lý cùng vật, một mẫu đổi thành chưa biết rồi lại đỏ. Có nên xóa cờ không?

Không nếu chưa xác nhận vùng trống. Mất nhãn tạm thời không chứng minh vật đã rời đi.

Tại sao chờ 1 s sau mỗi lần gạt chưa đủ chống lặp?

Vật có thể còn trước cảm biến hơn 1 s. Khi hết chờ, cùng vật sẽ bị nhận lại nếu không kiểm tra chu kỳ vào–ra.

Chọn độ sâu phù hợp

Mới bắt đầu · Chạy tay với cờ

Viết một ô đúng/sai cho cờ và cập nhật sau mỗi mẫu. Chưa cần lắp tay gạt; quan trọng là hiểu khi nào mở chu kỳ.

Chạy đỏ, đỏ, trống, đỏ và đánh dấu hai mẫu thực sự mở chu kỳ.

Đã học cơ bản · Theo dõi hai bộ đếm

Ghi riêng nhận vào và thành công để biết vật đã được nhận nhưng cơ cấu chưa hoàn tất. Không tăng thành công ngay khi chỉ vừa gửi lệnh motor.

Chạy hai vật, cho vật thứ hai bị kẹt; báo cả hai số và trạng thái chặn tiếp theo.

Muốn tìm hiểu sâu · Thiết kế tín hiệu cho phép nhận lại

Điều kiện trống cần xuất phát từ khả năng cảm biến hiện diện. Có thể yêu cầu ổn định nhiều mẫu để giảm dao động, nhưng phải thừa nhận lỗi có hệ thống vẫn tồn tại.

Đặc tả cờ được xóa khi nào, khi nào giữ, và hệ thống làm gì nếu tín hiệu hiện diện lỗi quá lâu.

Kiến thức trọng tâm

Mẫu đo và sự kiện là hai thứ khác nhau

Vòng lặp đọc 20 lần/s sẽ nhìn thấy 20 mẫu nếu một vật đứng trước cảm biến trong 1 s. Yêu cầu của hệ thống là một chu kỳ phân loại cho một vật, không phải một chu kỳ cho mỗi mẫu đỏ. Sự kiện "vật mới" cần được xác định bằng thay đổi hiện diện hoặc chu kỳ vào–ra, không chỉ bằng một giá trị màu.

Cờ đã xử lý làm bộ nhớ ngắn hạn

Ban đầu đã_xử_lý = sai. Khi có vật và nhãn hợp lệ, thực hiện một chu kỳ rồi đặt cờ đúng. Khi vật vẫn ở đó, dù đọc lại cùng màu cũng không khởi động lại. Chỉ đặt cờ sai khi xác nhận vùng đã trống. Nếu màu lúc rõ lúc chưa rõ nhưng vật còn nguyên, không được dùng nhãn chưa biết để tùy tiện đặt lại cờ.

Tách chốt chống lặp và báo thành công

Chốt đã xử lý nhằm ngăn lệnh lặp; bộ đếm thành công chỉ nên tăng khi chu kỳ gạt thực sự hoàn tất theo tiêu chí đặt trước. Nếu kẹt hoặc quá thời gian, ghi lỗi và chặn chu kỳ mới tới khi xử lý an toàn. Một delay cố định không thay thế phát hiện vật rời đi vì vật có thể đứng lâu hơn thời gian chờ.

Ví dụ có lời giải

Chuỗi hiện diện/màu: đỏ, đỏ, đỏ, trống, xanh, xanh, trống tạo hai vật mới. Bộ đếm chu kỳ tăng hai lần, không phải năm. Nếu chu kỳ gạt vật xanh bị lỗi, số vật vào là 2 nhưng số phân loại thành công chỉ là 1.

Lỗi thường gặp và cách tránh

1. Chờ 500 ms rồi đặt lại cờ không bảo đảm vật đã rời đi.

2. Màu chuyển từ đỏ sang chưa biết có thể là nhiễu của cùng một vật, không nhất thiết là một vật mới.

Các bước thực hành

1. Vẽ bảng từng mẫu với các cột nhãn, có vật?, cờ trước, hành động, cờ sau và bộ đếm.

2. Trong bàn màu, chọn đỏ rồi bấm Đọc nhiều lần. Dự đoán và xác nhận chỉ ghi một chu kỳ trước khi vật rời đi.

3. Chọn Không có vật rồi đọc để xác nhận vùng trống; đưa màu xanh vào và kiểm tra bộ đếm tăng đúng một lần.

4. Thêm trường hợp màu chưa xác định khi vật vẫn đứng yên và trường hợp kẹt trong bản thiết kế. Nêu rõ tín hiệu nào đủ để cho phép nhận vật tiếp.

Mã mẫu · Python · GearsBot Cage Bot (ev3dev2)

```
# GearsBot Cage Bot, ev3dev2 Python. Press Reset before Run.
from ev3dev2.motor import LargeMotor, OUTPUT_C, SpeedDPS
from ev3dev2.sensor.lego import ColorSensor, UltrasonicSensor
from time import time, sleep

try:
  color = ColorSensor("in3")
  sonar = UltrasonicSensor("in4")
  arm = LargeMotor(OUTPUT_C)
except Exception as error:
  print("SETUP:", error)
  raise RuntimeError("Select Cage Bot, then press Reset.")

def move_arm(target, deadline):
  if not 0 <= target <= 45:
    raise ValueError("Use an arm target from 0 to 45 degrees.")
  if time() >= deadline:
    return False
  limit = min(deadline, time() + 3.0)
  arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
  while abs(arm.position - target) > 2:
    if time() >= limit:
      arm.off(brake=True)
      return False
    sleep(0.02)
  arm.off(brake=True)
  return time() < limit

def read_group():
  samples = []
  for sample in range(3):
    samples.append(color.color)
    sleep(0.05)
  if samples[0] == samples[1] == samples[2]:
    if samples[0] == 5:  # EV3 red
      return "A"
    elif samples[0] == 2:  # EV3 blue
      return "B"
  return "CHECK"

NEAR_CM, CLEAR_CM = 3.0, 8.0
handled = False
clear_samples = 0
received = gate_cycles = needs_check = 0
deadline = time() + 90.0
try:
  if not move_arm(45, deadline):
    raise RuntimeError("Arm opening timeout; inspect and Reset.")
  print("READY: red=A, blue=B, other=CHECK")
  while time() < deadline:
    distance = sonar.distance_centimeters
    if distance <= 0:
      print("ERROR: invalid or contact-distance reading")
      break
    if distance >= CLEAR_CM:
      clear_samples += 1
      if clear_samples >= 3:
```

```
            handled = False
        else:
            clear_samples = 0

        if distance <= NEAR_CM and not handled:
            handled = True  # Latch before any motor command.
            received += 1
            group = read_group()
            print("object", received, "group", group)
            if group == "CHECK":
                needs_check += 1
            elif not (0 < sonar.distance_centimeters <= NEAR_CM):
                print("Object moved before gate cycle; keep latch set.")
            elif move_arm(0, deadline) and move_arm(45, deadline):
                gate_cycles += 1
                print("gate_cycles", gate_cycles)
            else:
                print("ERROR: gate timeout; inspect and Reset.")
                break
        sleep(0.05)
finally:
    arm.off(brake=True)
print("received", received, "gate_cycles", gate_cycles,
      "needs_check", needs_check)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn Robot → Select Robot → Cage Bot, chuyển sang Python rồi bấm Reset trước mỗi lần chạy. Mẫu dùng ev3dev2 của GearsBot: outC là tay chắn, in3 là cảm biến màu hướng trước, in4 là siêu âm; cổng A/B dành cho hai bánh xe. Chuẩn bị vật đỏ, xanh và vàng trên sân qua Worlds → World Builder; đặt mặt màu ngang tầm cảm biến. Mở joystick để điều khiển bánh xe: tiếp cận chậm đến khoảng cách ≤ 3 cm, dừng bánh, sau chu kỳ lùi ra ≥ 8 cm. Mốc 3 cm phù hợp tầm đọc 5 cm của cảm biến màu Cage Bot. Tay chắn khởi đầu ở 45°. Giữ một vật đỏ trong vùng đọc để xác nhận chỉ có một lượt nhận; với nhãn A/B, tay chắn hạ về 0° rồi mở lại 45°, có kiểm tra encoder và timeout 3 giây mỗi chặng. Vật vàng đi vào nhánh CHECK và vẫn chốt chống lặp. Chỉ sau ba mẫu khoảng cách ≥ 8 cm mới nhận vật mới. Số gate_cycles ghi chu kỳ tay chắn hoàn tất; nhóm A/B là kết quả nhận màu để đối chiếu, ngăn đích được ghi qua quan sát ở mô hình phân loại thực. Đây là mã chạy cho mô hình Cage Bot; với Hub SPIKE thật, dùng API của ứng dụng LEGO Education SPIKE và cấu hình cổng của bộ lắp.

Bài tập áp dụng

Chạy tay chuỗi đỏ, đỏ, trống, đỏ, chưa biết, trống. Giả sử "chưa biết" vẫn có cùng vật thứ hai và hai chu kỳ đều gạt thành công. Có bao nhiêu vật mới? Sau đó đổi chu kỳ thứ hai thành kẹt, báo lại hai loại bộ đếm.

Gợi ý đối chiếu sau khi tự làm

Có hai vật mới: đỏ đầu và đỏ sau lần trống. Mẫu chưa biết không mở chu kỳ mới. Nếu cả hai thành công: vào 2, thành công 2. Nếu vật thứ hai kẹt: vào 2, thành công 1, lỗi 1; cờ chống lặp vẫn không được xóa chỉ vì đọc lại nhãn không rõ.

Tự kiểm tra mức hiểu

1. Theo dõi được cờ trước/sau từng mẫu của chuỗi có ít nhất hai vật.

2. Nêu được điều kiện đặt lại cờ mà không đánh đồng chưa biết với trống.

3. Báo cáo riêng số vật nhận vào và số chu kỳ gạt thành công khi có lỗi.

Câu hỏi trắc nghiệm

Để tránh gạt lặp cho cùng một vật cần...

1. Tăng tốc vòng lặp

2. Ghi nhận trạng thái đã xử lý

3. Luôn quay 90°

Đáp án: 2. Cờ đã xử lý lưu việc vật hiện tại đã kích hoạt một chu kỳ. Tăng tốc vòng lặp tạo thêm mẫu chứ không tự phân biệt được vật mới với vật cũ.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

GearsBot — cấu hình Cage Bot chính thức

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GearsBot — Python ev3dev2 motor API

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GearsBot — Python ev3dev2 cảm biến màu và siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 4/6 · Dự án: trạm phân loại

Đặt yêu cầu kiểm thử cho trạm phân loại với tập vật đã xác định trước.

Chuẩn bị

Hoàn thành cơ cấu, đọc màu và chống đếm lặp. Biết tính phân số thành phần trăm và lập bảng kỳ vọng–thực tế.

Sau bài học, bạn có thể

1. Đặt yêu cầu kiểm thử cho trạm phân loại với tập vật đã xác định trước.

2. Tính tỷ lệ đúng trên toàn bộ lượt thử và báo cáo riêng kẹt/chưa xác định.

3. Đề xuất một cải tiến từ dữ liệu và kiểm tra lại trong điều kiện tương đương.

Cùng hiểu bài từ đầu

Trạm phân loại có nhiều khâu: thấy vật, nhận màu, chọn ngăn, điều khiển tay gạt và đưa vật tới ngăn. Sản phẩm chỉ đạt khi đầu ra cuối đúng theo yêu cầu, không chỉ khi một khâu báo tốt. Bài dự án giúp bạn đặt tiêu chí trước, giữ mọi lượt thử trong báo cáo và dùng dữ liệu để biết nên sửa ở đâu.

Một đơn hàng được giao thành công khi đến đúng người; đọc đúng địa chỉ ở kho mới hoàn thành một phần công việc.

Tương tự, nhận đúng màu là một khâu. Vật phải tới đúng ngăn thì lượt phân loại mới đúng theo yêu cầu toàn trạm.

Giới hạn của ví von: Ví dụ không thay thế tiêu chí của dự án. Với vật ngoài tập đỏ/xanh, bạn phải định nghĩa trước ngăn kiểm tra hay cách xử lý khác mới chấm được đúng/sai.

Những từ cần hiểu

1. Tiêu chí đầu ra: Mô tả kết quả có thể quan sát, như đỏ vào A, xanh vào B, loại khác vào ngăn kiểm tra. Cần chốt trước khi chạy.

2. Tỷ lệ đúng: Số lượt có đầu ra đúng chia tổng số lượt thử, rồi nhân 100%. Các lượt kẹt, timeout hoặc không hoàn tất vẫn nằm trong tổng.

3. Nguyên nhân lỗi: Khâu có bằng chứng gây đầu ra sai, như nhãn đo sai, gạt sai góc hoặc vật mắc. Đầu ra sai riêng nó chưa chỉ ra nguyên nhân.

4. Biến kiểm soát: Các điều giữ giống nhau để so sánh, như bộ vật, thứ tự, khoảng đặt, ánh sáng và tốc độ. Chỉ đổi yếu tố đang muốn khảo sát.

Tính tỷ lệ mà không làm mất lượt lỗi

Bắt đầu với: Bộ thử 12 lượt đã được xác định tiêu chí. Kết quả: 9 đúng ngăn, 1 sai ngăn, 1 chưa xác định không hoàn tất, 1 kẹt.

Chốt mẫu số

Có 12 lượt đưa vào theo bộ thử, nên mẫu số là 12. Không bỏ hai lượt chưa hoàn tất ra khỏi báo cáo.

Tính tỷ lệ

Số lượt đúng là 9. Tỷ lệ đúng = 9/12 × 100% = 75% trên bộ thử này.

Tìm dữ liệu còn thiếu

Với lượt sai ngăn, cần nhãn đã đọc và lệnh gạt để phân biệt cảm biến sai với cơ cấu sai. Với lượt kẹt, ghi vị trí và thời gian phát hiện.

Chọn một cải tiến

Nếu bằng chứng cho thấy khoảng đặt dao động gây nhãn chưa biết, thêm cách giữ khoảng đặt rồi thử lại cùng bộ vật. Ghi cả tỷ lệ và loại lỗi sau sửa.

Kết quả: Báo cáo: 12 lượt, 9 đúng, 1 sai ngăn, 1 chưa hoàn tất do chưa xác định, 1 kẹt; tỷ lệ đúng 75%.

Mẫu số đầy đủ phản ánh trải nghiệm toàn quy trình. Phân loại lỗi theo khâu giúp sửa có cơ sở; 75% không phải bảo đảm cho mọi vật hay mọi ánh sáng ngoài bộ đã thử.

Tự giải thích rồi đối chiếu

Nếu cảm biến nhận đúng 10/10 nhưng chỉ 8 vật tới đúng ngăn, tỷ lệ đúng toàn trạm có phải 100% không?

Không. Theo tiêu chí đến đúng ngăn, tỷ lệ là 8/10 = 80%; cần xem cơ cấu và điều khiển các lượt còn lại.

Có được đổi tiêu chí vật lạ sau khi thấy trạm xử lý chúng không?

Không trong cùng lượt đánh giá. Tiêu chí phải đặt trước; nếu thay yêu cầu cần ghi phiên bản và thử lại.

Chọn độ sâu phù hợp

Mới bắt đầu · Theo một vật từ đầu đến cuối

Đánh dấu vật kỳ vọng vào ngăn nào, nhãn đọc là gì và cuối cùng nằm ở đâu. Bắt đầu với từng vật, chưa cần chạy nhanh.

Lập ba cột màu chuẩn bị, nhãn đọc, ngăn thực cho đỏ/xanh/khác.

Đã học cơ bản · Ghi lỗi theo khâu

Tách nhận màu, quyết định nhánh và cơ cấu. Nhờ vậy một con số tỷ lệ chung có thể dẫn tới hành động sửa cụ thể.

Với một lượt nhận đỏ nhưng rơi vào B, liệt kê dữ liệu cần để kiểm tra phần điều khiển và phần cơ khí.

Muốn tìm hiểu sâu · So sánh cải tiến có phạm vi

Giữ bộ thử và quy tắc chấm, thay một yếu tố rồi thử lại. Nếu đã dùng bộ thử để chỉnh nhiều lần, nên có thêm bộ mới để đánh giá khả năng áp dụng.

Thiết kế thử A/B cho hai khoảng đặt: ghi điều giữ nguyên, cách đếm mọi lượt lỗi và giới hạn kết luận từ kết quả.

Kiến thức trọng tâm

Đầu ra đúng phải được định nghĩa trước

Ví dụ đỏ vào ngăn A, xanh vào B, vật ngoài hai loại vào ngăn kiểm tra. Một lượt đúng là vật tới đúng vị trí theo yêu cầu, không chỉ cảm biến đọc đúng màu. Trước khi thử, ghi số vật, thứ tự vào, điều kiện sáng, khoảng đặt, tốc độ và tiêu chí kẹt/timeout để người khác có thể lặp lại.

Độ đúng của toàn quy trình

Tỷ lệ đúng = số lượt có đầu ra đúng / tổng số lượt thử × 100%. Mẫu số gồm mọi vật đưa vào theo đề bài, kể cả vật chưa xác định, kẹt hoặc không hoàn tất. Đồng thời nên báo riêng số nhận dạng sai, số gạt sai và số kẹt để biết khâu cần sửa. Không loại lỗi khỏi mẫu số để tăng điểm.

Cải tiến có kiểm chứng

Sau lượt thử ban đầu, chọn một nguyên nhân có bằng chứng, ví dụ khoảng cách cảm biến dao động hoặc tay gạt không về nghỉ. Chỉ đổi một yếu tố trước khi thử lại cùng bộ vật và điều kiện. Kết quả 8/10 chỉ mô tả bộ thử đó; chưa đủ để khẳng định máy luôn đúng 80% cho mọi vật, ánh sáng và tốc độ.

Ví dụ có lời giải

10 vật được đưa vào: 8 tới đúng ngăn, 1 vào sai ngăn, 1 kẹt. Tỷ lệ đúng toàn hệ thống là 8/10 × 100% = 80%, không phải 8/9. Nếu cảm biến nhận đúng cả 10 nhưng cơ cấu lỗi 2, cần sửa cơ khí/điều khiển chứ không chỉ thay ngưỡng nhận màu.

Lỗi thường gặp và cách tránh

1. Một vật được nhận đúng màu nhưng gạt sai ngăn vẫn là lỗi đầu ra của toàn hệ thống.

2. Tỷ lệ của một tập thử nhỏ không phải bảo đảm xác suất đúng ở mọi tình huống mới.

Các bước thực hành

1. Chốt ánh xạ màu–ngăn và chuẩn bị 10 lượt vật gồm đỏ, xanh, khác. Viết sẵn cột kỳ vọng; không sửa kỳ vọng sau khi chạy.

2. Chạy bản logic nội bộ, lấy vật ra giữa các lượt và ghi nhánh/bộ đếm. Phân biệt kết quả logic với thử cơ cấu thật.

3. Nếu có kit: kiểm tra tay gạt/gốc/góc an toàn trước, chạy từng vật và ghi màu đo, ngăn thực, kẹt hoặc timeout.

4. Tính tỷ lệ đúng với đầy đủ 10 lượt; chọn một sửa đổi từ bảng lỗi, thử lại và so sánh cả tỷ lệ đúng lẫn lỗi còn lại.

Mã mẫu · Python · GearsBot Cage Bot (ev3dev2)

```
# GearsBot Cage Bot, ev3dev2 Python. Press Reset before Run.
from ev3dev2.motor import LargeMotor, OUTPUT_C, SpeedDPS
from ev3dev2.sensor.lego import ColorSensor, UltrasonicSensor
from time import time, sleep

try:
  color = ColorSensor("in3")
  sonar = UltrasonicSensor("in4")
  arm = LargeMotor(OUTPUT_C)
except Exception as error:
  print("SETUP:", error)
  raise RuntimeError("Select Cage Bot, then press Reset.")

def move_arm(target, deadline):
  if not 0 <= target <= 45:
    raise ValueError("Use an arm target from 0 to 45 degrees.")
  if time() >= deadline:
    return False
  limit = min(deadline, time() + 3.0)
  arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
  while abs(arm.position - target) > 2:
    if time() >= limit:
      arm.off(brake=True)
      return False
    sleep(0.02)
  arm.off(brake=True)
  return time() < limit

def read_group():
  samples = []
  for sample in range(3):
    samples.append(color.color)
    sleep(0.05)
  if samples[0] == samples[1] == samples[2]:
    if samples[0] == 5:  # EV3 red
      return "A"
    elif samples[0] == 2:  # EV3 blue
      return "B"
  return "CHECK"

def wait_zone(near, deadline):
  stable = 0
  while time() < deadline:
    distance = sonar.distance_centimeters
    if distance <= 0:
      return False
    matches = distance <= 3.0 if near else distance >= 8.0
    stable = stable + 1 if matches else 0
    if stable >= 3:
      return True
    sleep(0.05)
  return False

# Prepare objects in this exact order. CHECK uses a yellow object.
expected_groups = ["A", "B", "CHECK", "A", "B",
           "A", "CHECK", "B", "A", "B"]
results = []
```

```
recognized = passed = 0
motor_fault = False
end_time = time() + 240.0
try:
  if not move_arm(45, end_time):
    raise RuntimeError("Arm opening timeout; inspect and Reset.")
  for index in range(len(expected_groups)):
    expected = expected_groups[index]
    observed, status = "NO_READING", "WAIT_CLEAR_TIMEOUT"
    print("Trial", index + 1, "expected", expected,
        "reverse until clear, then approach the next object")
    if wait_zone(False, min(end_time, time() + 10.0)):
      print("APPROACH: 20 seconds")
      status = "OBJECT_TIMEOUT"
      if wait_zone(True, min(end_time, time() + 20.0)):
        observed = read_group()
        status = "CHECK" if observed == "CHECK" else "OBJECT_MOVED"
        if observed != "CHECK" and 0 < sonar.distance_centimeters <= 3.0:
          if move_arm(0, end_time) and move_arm(45, end_time):
            status = "GATE_OK"
          else:
            status = "GATE_TIMEOUT"
            motor_fault = True
    recognition_ok = observed == expected
    protocol_ok = recognition_ok and (
      status == "GATE_OK" or (expected == "CHECK" and status == "CHECK"))
    recognized += int(recognition_ok)
    passed += int(protocol_ok)
    results.append([index + 1, expected, observed, status])
    print("RESULT", results[-1], "pass", protocol_ok)
    if motor_fault or time() >= end_time:
      print("STOP: inspect the robot before another run.")
      break
finally:
  arm.off(brake=True)

attempted = len(results)  # Includes every timeout and incorrect result.
print("planned", len(expected_groups), "attempted", attempted)
if attempted:
  print("recognition_percent", round(100.0 * recognized / attempted, 1))
  print("classification_and_gate_percent", round(100.0 * passed / attempted, 1))
print("complete_test", attempted == len(expected_groups))
print("These results measure labels and arm cycles; record destination bins separately.")
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn Robot → Select Robot → Cage Bot, chuyển sang Python rồi bấm Reset trước mỗi lần chạy. Mẫu dùng ev3dev2 của GearsBot: outC là tay chắn, in3 là cảm biến màu hướng trước, in4 là siêu âm; cổng A/B dành cho hai bánh xe. Chuẩn bị vật đỏ, xanh và vàng trên sân qua Worlds → World Builder; đặt mặt màu ngang tầm cảm biến. Mở joystick để điều khiển bánh xe: tiếp cận chậm đến khoảng cách ≤ 3 cm, dừng bánh, sau chu kỳ lùi ra ≥ 8 cm. Mốc 3 cm phù hợp tầm đọc 5 cm của cảm biến màu Cage Bot. Chuẩn bị thứ tự 10 lượt đỏ, xanh, vàng, đỏ, xanh, đỏ, vàng, xanh, đỏ, xanh. Theo thông báo Trial/APPROACH, lùi để vùng trống ổn định rồi tiếp cận vật kế tiếp; giữ vật đến khi xuất RESULT. Mỗi lượt chờ trống tối đa 10 giây, chờ vật 20 giây; toàn bài tối đa 240 giây. Mã ghi kỳ vọng, nhãn đo, timeout và kết quả tay chắn; mọi lượt đã bắt đầu đều ở mẫu số, kể cả lỗi. Nếu dừng sớm, planned/attempted và complete_test thể hiện rõ số lượt đã làm. Tỷ lệ ở đây đo nhận dạng và chu kỳ tay chắn Cage Bot. Với trạm SPIKE có ngăn A/B, ghi thêm ngăn quan sát được và tính độ đúng đầu ra trên toàn bộ lượt đã thử. Đây là mã chạy cho mô hình Cage Bot; với Hub SPIKE thật, dùng API của ứng dụng LEGO Education SPIKE và cấu hình cổng của bộ lắp.

Bài tập áp dụng

Một hệ thống thử 12 vật: 9 đúng ngăn, 1 sai ngăn, 1 chưa xác định, 1 kẹt. Tính tỷ lệ đúng. Nêu dữ liệu cần thêm để biết nên sửa cảm biến hay cơ cấu, và thiết kế phép thử tiếp theo chỉ thay một yếu tố.

Gợi ý đối chiếu sau khi tự làm

Tỷ lệ đúng = 9/12 = 75%. Ghi nhãn cảm biến và vị trí vật thực cho từng lượt để tách nhận dạng sai khỏi gạt sai. Có thể giữ cơ khí/tốc độ/bộ vật, chỉ cố định lại khoảng cách cảm biến rồi thử lại. Không bỏ lượt chưa biết/kẹt khỏi mẫu số.

Tự kiểm tra mức hiểu

1. Định nghĩa đầu ra đúng có cả ngăn đích, không chỉ nhãn màu.

2. Tính được tỷ lệ đúng với đầy đủ các lượt lỗi trong mẫu số.

3. Đề xuất được một phép thử thay đúng một yếu tố và nêu dữ liệu sẽ thu.

Câu hỏi trắc nghiệm

8/10 vật phân loại đúng tương ứng...

1. 8%

2. 80%

3. 100%

Đáp án: 2. 8/10 = 0,8; nhân 100% được 80%. Đây là tỷ lệ của tập thử đã cho; cách tính cần bao gồm các lượt lỗi theo tiêu chí đặt trước.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

GearsBot — cấu hình Cage Bot chính thức

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GearsBot — Python ev3dev2 motor API

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GearsBot — Python ev3dev2 cảm biến màu và siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 5/6 · Bánh răng: tốc độ và mô-men

Tính tỷ số tốc độ và góc từ số răng của một cặp bánh răng ngoài.

Chuẩn bị

Biết tỷ lệ, vòng/phút (rpm), góc quay và ý nghĩa định tính của mô-men. Xác định được bánh chủ động và bánh bị động.

Sau bài học, bạn có thể

1. Tính tỷ số tốc độ và góc từ số răng của một cặp bánh răng ngoài.

2. Giải thích mối đánh đổi giữa tốc độ và mô-men khi bỏ qua tổn hao.

3. Nêu giới hạn của phép tính lý tưởng và chọn bộ truyền theo yêu cầu cơ cấu.

Cùng hiểu bài từ đầu

Bộ truyền bánh răng giúp chọn cách dùng chuyển động của motor. Bánh nhỏ kéo bánh lớn làm đầu ra chậm hơn trong mô hình lý tưởng, đồng thời tăng mô-men; đảo cặp bánh thì đổi theo hướng ngược lại. Để hiểu điều này, ta cần nói rõ đầu vào, đầu ra, số răng, đơn vị tốc độ và chiều quay. Bộ truyền không sinh thêm năng lượng miễn phí.

Quan sát răng tại chỗ ăn khớp: mỗi răng đi qua của bánh vào phải tương ứng với một răng của bánh ra.

Bánh ra có ba lần số răng cần ba lần lượng răng đi qua để quay đủ một vòng; vì thế nó quay bằng một phần ba tốc độ bánh vào.

Giới hạn của ví von: Phép đếm răng giải thích tỷ số hình học. Tốc độ có tải và mô-men thực còn chịu tổn hao, giới hạn motor và độ bền cơ cấu; không suy chúng chính xác từ ảnh mô phỏng.

Những từ cần hiểu

1. rpm: Số vòng quay trong một phút. Có thể dùng rpm ở cả hai phía khi tính tỷ số tốc độ, nhưng khi tính công suất bằng watt cần tốc độ góc theo rad/s.

2. Mô-men: Đại lượng thể hiện tác dụng làm quay của lực quanh trục, đo bằng N·m. Nó khác tốc độ quay và khác một lực đơn lẻ.

3. Tỷ số truyền: Trong bài dùng độ lớn tốc độ ra/vào = số răng vào/số răng ra. Nêu quy ước này vì tài liệu khác có thể gọi tỷ số theo chiều ngược.

4. Hiệu suất η: Tỷ phần công suất hữu ích còn lại sau tổn hao, nằm dưới hoặc bằng 1 trong mô hình. η = 0,8 nghĩa là 80%, không phải thêm 80% năng lượng.

Giảm tốc với cặp 12:36

Bắt đầu với: Bánh vào 12 răng, bánh ra 36 răng, tốc độ vào 90 rpm, mô-men vào giả định 0,10 N·m; xét mô hình hiệu suất 0,8.

Tính tốc độ

Độ lớn tốc độ ra = 90 × 12/36 = 30 rpm. Hai bánh răng ngoài ăn khớp nên chiều ra ngược chiều vào.

Tính mô-men lý tưởng

Tỷ số mô-men lý tưởng là 36/12 = 3. Mô-men ra lý tưởng = 0,10 × 3 = 0,30 N·m.

Thêm tổn hao trong mô hình

Với η = 0,8, mô-men ra xấp xỉ 0,8 × 0,30 = 0,24 N·m ở điều kiện mà mô hình áp dụng. Không coi đây là số đo motor thật.

Liên hệ nhiệm vụ

Đầu ra chậm hơn có thể phù hợp tay gạt cần mô-men hơn. Nhưng vẫn phải kiểm tra tải, góc, thời gian và giới hạn cơ khí thực tế.

Kết quả: Đầu ra lý tưởng về tốc độ: 30 rpm ngược chiều. Mô-men lý tưởng 0,30 N·m; mô hình có hiệu suất 0,8 cho khoảng 0,24 N·m.

Trong lý tưởng, công suất được bảo toàn: giảm tốc đổi lấy tăng mô-men. Với tổn hao, công suất ra giảm; bộ truyền không cho phép vượt các giới hạn của motor hoặc bánh răng.

Tự giải thích rồi đối chiếu

Đảo thành bánh vào 36, ra 12 ở 60 rpm thì tốc độ ra bao nhiêu?

Độ lớn 60 × 36/12 = 180 rpm, ngược chiều. Mô-men ra lý tưởng chỉ bằng một phần ba đầu vào.

Thêm một bánh trung gian đơn giản có tự đổi tỷ số đầu/cuối không?

Không nếu đó chỉ là một bánh trung gian, không ghép tầng đồng trục; nó đổi chiều thêm một lần. Tỷ số cuối vẫn phụ thuộc số răng đầu và cuối.

Chọn độ sâu phù hợp

Mới bắt đầu · Đếm răng để đoán nhanh/chậm

Chỉ hai bánh, chỉ rõ bánh nào được motor kéo. Bánh ra nhiều răng hơn sẽ quay ít vòng hơn khi cùng số răng đi qua điểm ăn khớp.

So sánh 12:36 và 36:12, dự đoán đầu ra nhanh hay chậm trước khi tính.

Đã học cơ bản · Nối góc và tốc độ

Cùng một tỷ số răng áp dụng cho độ lớn góc và tốc độ trong mô hình cứng. Góc motor 90° ở cặp 12:36 cho đầu ra 30° ngược chiều.

Tính đồng thời góc ra với đầu vào 180° và tốc độ ra với đầu vào 60 rpm.

Muốn tìm hiểu sâu · Kiểm tra công suất và tổn hao

P = Tω dùng T theo N·m và ω theo rad/s để ra W. Đổi rpm sang rad/s bằng nhân 2π/60; tốc độ và mô-men cần thuộc cùng điều kiện vận hành.

Dùng số trong ví dụ để kiểm tra công suất lý tưởng vào/ra rồi giải thích ảnh hưởng của η = 0,8.

Kiến thức trọng tâm

Tỷ số tốc độ và chiều quay

Với hai bánh răng ngoài ăn khớp, độ lớn tốc độ ra/vào = Zvào/Zra. Tốc độ góc có dấu thỏa ωra = −ωvào × Zvào/Zra vì hai trục quay ngược chiều. Có thể dùng cùng một đơn vị rpm ở cả hai phía khi tính tỷ số. Một bánh trung gian đơn giản đổi chiều thêm một lần; tỷ số cuối vẫn phụ thuộc bánh đầu/cuối nếu không ghép nhiều tầng đồng trục.

Giảm tốc đổi lấy mô-men

Trong mô hình lý tưởng không tổn hao, công suất P = Tω được bảo toàn; T là mô-men, ω phải theo rad/s khi tính P bằng watt. Vì thế độ lớn Tra/Tvào = Zra/Zvào. Với hiệu suất η < 1, mô-men ra chỉ xấp xỉ η × Tvào × Zra/Zvào. Bộ truyền không tạo năng lượng và không cho phép vượt giới hạn cơ khí.

Chọn tỷ số theo nhiệm vụ

Bánh nhỏ kéo bánh lớn làm giảm tốc và thường tăng mô-men ra; phù hợp khi tay gạt cần lực hơn tốc độ. Đổi ngược cặp bánh làm tăng tốc nhưng giảm mô-men lý tưởng. Tải, ma sát, độ rơ và giới hạn motor khiến tốc độ thực khác phép tính không tải, nên phải đo ở điều kiện sử dụng.

Ví dụ có lời giải

Bánh vào 12 răng quay 90 rpm kéo bánh ra 36 răng: độ lớn tốc độ ra = 90 × 12/36 = 30 rpm và ngược chiều. Nếu mô-men vào 0,10 N·m, mô-men ra lý tưởng 0,30 N·m; với η = 0,8 thì khoảng 0,24 N·m theo mô hình đơn giản.

Lỗi thường gặp và cách tránh

1. Tăng mô-men nhờ giảm tốc không tạo thêm năng lượng; tốc độ bị đổi và luôn có tổn hao thực tế.

2. Phần trăm tốc độ trong ứng dụng không tự bằng rpm nếu chưa biết đặc tính motor và tải.

Các bước thực hành

1. Vẽ hai bánh, đánh dấu đầu vào/đầu ra và chiều quay. Đếm răng thực thay vì suy từ đường kính nhìn bằng mắt.

2. Tính tốc độ ra khi đầu vào 90 rpm cho cặp 12:36, rồi đảo thành 36:12. Viết cả chiều quay.

3. Dùng công thức để dự đoán góc tay gạt cần; với cơ cấu thật, kiểm tra góc giới hạn trước khi ra lệnh motor.

4. Nếu có kit, đo thời gian/góc chuyển động ở cùng tải; ghi chênh lệch với dự đoán và xem xét ma sát/độ rơ. Không suy mô-men đo được chỉ từ hình ảnh mô phỏng.

Mã mẫu · Python 3 · tính tỷ số truyền

```
# Python 3: one external gear pair, not a measurement from a motor.
teeth_in = 12
teeth_out = 36
speed_in_rpm = 90.0
torque_in_nm = 0.10
efficiency = 0.80

if teeth_in <= 0 or teeth_out <= 0:
  raise ValueError("Tooth counts must be positive.")
if not 0 < efficiency <= 1:
  raise ValueError("Efficiency must be between 0 and 1.")

speed_out_rpm = -speed_in_rpm * teeth_in / teeth_out
torque_out_nm = efficiency * torque_in_nm * teeth_out / teeth_in
print("speed_out_rpm", speed_out_rpm)  # -30.0
print("torque_out_nm", round(torque_out_nm, 3))  # 0.24
```

Chọn và sử dụng phòng lab

Chuyển editor GearsBot sang Python, dán mã và chạy để nhận tốc độ đầu ra −30 rpm và mô-men ước lượng 0,24 N·m. Đổi cặp răng 12:36 thành 36:12 rồi so sánh kết quả; dấu âm biểu thị đảo chiều của một cặp bánh răng ngoài. Đây là phép tính mô hình với hiệu suất đã nhập. Khi đo bộ truyền thật, ghi số răng, tải, tốc độ và hiệu suất ước lượng để đối chiếu với kết quả.

Bài tập áp dụng

Đầu vào 60 rpm, bánh vào 36 răng, bánh ra 12 răng: tính tốc độ/chiều ra và tỷ số mô-men lý tưởng. So sánh với cặp 12:36 để chọn cho một cơ cấu cần quay chậm và gạt vật nặng hơn.

Gợi ý đối chiếu sau khi tự làm

Độ lớn tốc độ ra 60 × 36/12 = 180 rpm, ngược chiều; mô-men lý tưởng bằng 1/3 mô-men vào. Cặp 12:36 giảm tốc còn 1/3 và tăng mô-men lý tưởng gấp 3, phù hợp hơn về xu hướng cho yêu cầu chậm/lực lớn; vẫn phải kiểm tra tải, hiệu suất và giới hạn cơ khí.

Tự kiểm tra mức hiểu

1. Xác định đúng tỷ số Zvào/Zra thay vì đảo ngược.

2. Nêu được cả tốc độ và chiều quay cho cặp 12:36.

3. Giải thích được vì sao mô-men thật có thể thấp hơn giá trị lý tưởng.

Câu hỏi trắc nghiệm

12 răng kéo 36 răng, đầu vào 90 vòng/phút thì đầu ra...

1. 270

2. 90

3. 30

Đáp án: 3. 90 × 12/36 = 30 vòng/phút về độ lớn. Vì là một cặp bánh răng ngoài, chiều quay bị đảo; các lựa chọn trắc nghiệm chỉ nêu độ lớn tốc độ.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm
2. Thực hành đúng quy trình: 30 điểm
3. Dữ liệu và kiểm thử: 30 điểm
4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

Bài 6/6 · Máy trạng thái cho bộ phân loại

Thiết kế máy trạng thái hữu hạn có điều kiện chuyển rõ ràng cho một chu kỳ phân loại.

Chuẩn bị

Hoàn thành bài cờ chống đếm lặp và dự án phân loại. Biết điều kiện Boolean, bộ đếm và khái niệm giới hạn thời gian.

Sau bài học, bạn có thể

1. Thiết kế máy trạng thái hữu hạn có điều kiện chuyển rõ ràng cho một chu kỳ phân loại.

2. Tách trạng thái chờ vật, phân loại, chờ trống và lỗi để ngăn kích hoạt lặp.

3. Kiểm thử chuỗi sự kiện bình thường, màu chưa biết và kẹt/quá thời gian.

Cùng hiểu bài từ đầu

Khi hệ thống vừa phải chờ vật, gạt, chờ vật rời và xử lý kẹt, một điều kiện "nếu đỏ thì gạt" không còn đủ rõ. Máy trạng thái đặt tên cho việc đang làm và chỉ cho phép những chuyển tiếp nhất định. Mỗi mẫu cảm biến được diễn giải cùng trạng thái hiện tại, nên cùng nhãn đỏ có thể mở chu kỳ mới ở WAIT nhưng không mở chu kỳ ở WAIT_CLEAR.

Hãy nghĩ đến các bước mượn sách: chờ bạn đọc, xử lý mượn, chờ hoàn tất bàn giao rồi mới phục vụ lượt tiếp.

Trạng thái giữ ngữ cảnh để một yêu cầu lặp không bị coi là người mới. Các mũi tên chỉ ra sự kiện nào cho phép chuyển bước.

Giới hạn của ví von: Robot cần điều kiện và thời hạn đo được; không thể dựa vào phán đoán mơ hồ như "có vẻ xong". Trạng thái ERROR còn cần quy tắc dừng và phục hồi rõ ràng.

Những từ cần hiểu

1. Trạng thái: Tên của một chế độ hiện tại, như WAIT, CLASSIFY, WAIT_CLEAR, ERROR. Nó giúp quyết định đầu vào nào có ý nghĩa ở thời điểm đó.

2. Điều kiện chuyển: Sự kiện hoặc điều kiện cho phép sang trạng thái khác, như xác nhận có vật, gạt hoàn tất hoặc quá thời gian.

3. Hành động khi vào: Việc khởi tạo một lần lúc vừa vào trạng thái, chẳng hạn bắt đầu gạt. Khác với gửi lại lệnh gạt ở mọi vòng lặp.

4. Timeout: Giới hạn thời gian đã đặt. Quá giới hạn mà chưa hoàn tất phải đi nhánh lỗi/dừng phù hợp, không được mặc nhiên báo thành công.

Một nhãn chưa biết trong lúc chờ vật rời

Bắt đầu với: Trạm đang WAIT_CLEAR sau khi đã gạt một vật đỏ. Mẫu tiếp theo: đỏ, chưa biết, đỏ, trống đã xác nhận, đỏ mới.

Ba mẫu đầu

Đỏ và nhãn chưa biết chưa xác nhận vật rời. Trạm giữ WAIT_CLEAR; không chạy lại gạt và không tăng bộ đếm nhận vật.

Mẫu trống đáng tin

Điều kiện vùng trống được xác nhận theo thiết kế, nên chuyển WAIT_CLEAR → WAIT. Lúc này mới cho phép vật tiếp theo.

Đỏ cuối

Ở WAIT, có vật mới nên tăng nhận vào và sang CLASSIFY. Khởi tạo gạt đúng một lần khi vào, không khởi tạo lại với mỗi mẫu đỏ.

Xét hoàn tất hoặc lỗi

Nếu gạt hoàn tất thì tăng thành công, sang WAIT_CLEAR. Nếu quá hạn thì dừng motor và sang ERROR, không tăng thành công; cần xử lý lỗi trước khi nhận tiếp.

Kết quả: Chỉ mẫu đỏ cuối mở một chu kỳ mới. Các mẫu đỏ trước và nhãn chưa biết không làm mất ngữ cảnh của vật cũ.

Cùng đầu vào có thể dẫn đến hành động khác vì trạng thái khác. Quy tắc chuyển và hành động một lần làm hệ thống có thể kiểm tra, đồng thời tách nhận vật khỏi xác nhận thành công.

Tự giải thích rồi đối chiếu

CLASSIFY bị timeout có được tự quay lại WAIT để thử vật khác ngay không?

Không theo thiết kế an toàn của bài. Phải dừng và sang ERROR, xác định cách phục hồi để vật kẹt không gây chu kỳ lặp.

Nhiều mẫu “chưa biết” liên tiếp có chắc là trống không?

Không. Nếu cảm biến không phân biệt được mất nhãn và không có vật, lặp mẫu không tự tạo bằng chứng hiện diện đáng tin.

Chọn độ sâu phù hợp

Mới bắt đầu · Đóng vai từng trạng thái

Viết bốn thẻ tên trạng thái. Đặt một thẻ hiện tại, đọc một mẫu rồi chỉ chuyển khi điều kiện trên mũi tên đúng.

Chạy đỏ, đỏ, trống, xanh, trống với giả định mỗi chu kỳ hoàn tất; đánh dấu hai lần vào CLASSIFY.

Đã học cơ bản · Làm bảng chuyển đầy đủ

Với mỗi trạng thái, ghi đầu vào liên quan, trạng thái kế tiếp và hành động. Phân biệt giữ nguyên trạng thái với khởi tạo một chu kỳ mới.

Ở WAIT_CLEAR, ghi ba trường hợp có vật, chưa xác nhận, trống xác nhận và hành động của từng trường hợp.

Muốn tìm hiểu sâu · Đặc tả thời gian và phục hồi

Ghi thời điểm vào CLASSIFY và so thời gian đã trôi với giới hạn. ERROR phải có điều kiện phục hồi được kiểm chứng, ví dụ can thiệp rồi kiểm tra vùng/cơ cấu, không chỉ hết một delay.

Thiết kế một ca gạt kẹt, một ca mất nhãn tạm và một ca phục hồi; báo cả số nhận vào lẫn số thành công.

Kiến thức trọng tâm

Trạng thái mô tả hệ thống đang làm gì

Máy trạng thái hữu hạn gồm một tập trạng thái, đầu vào/sự kiện và quy tắc chuyển. Bản tối thiểu có WAIT chờ vật, CLASSIFY xử lý đúng một lần, WAIT_CLEAR chờ vật rời đi. Bổ sung ERROR khi cơ cấu kẹt hoặc số đo cần thiết bị lỗi; phải nêu cách phục hồi an toàn thay vì tự lặp vô hạn.

Chuyển trạng thái khác với chạy hành động liên tục

Khi WAIT thấy vật mới, ghi một lượt nhận vật rồi sang CLASSIFY. Hành động gạt được khởi tạo một lần khi vào CLASSIFY; không khởi tạo lại mỗi vòng lặp. Khi hoàn tất thành công, tăng đếm thành công rồi sang WAIT_CLEAR. Nhãn không rõ được đưa tới vùng kiểm tra theo yêu cầu, hoặc yêu cầu can thiệp; không tự gán đỏ.

Timeout và điều kiện trống đáng tin cậy

Nếu CLASSIFY quá thời gian mà chưa hoàn tất, dừng motor và sang ERROR. Trong WAIT_CLEAR chỉ về WAIT khi xác nhận vùng cảm biến trống theo tiêu chí đã đặt. Có thể yêu cầu trống ổn định nhiều mẫu để giảm nhiễu, nhưng nhiều mẫu liên tiếp không tự biến một phép đo kém tin cậy thành bằng chứng chắc chắn; phải đánh giá cảm biến hiện diện thực.

Ví dụ có lời giải

Với chuỗi đỏ, đỏ, trống, xanh, trống và giả sử mỗi chu kỳ hoàn tất: WAIT → CLASSIFY → WAIT_CLEAR; mẫu đỏ thứ hai không tạo chu kỳ mới; trống đưa về WAIT; xanh tạo chu kỳ thứ hai. Nhận vào 2, phân loại thành công 2. Nếu tay gạt bị kẹt khi xanh, trạng thái phải sang ERROR và không tăng thành công.

Lỗi thường gặp và cách tránh

1. Viết tên trạng thái không tự bảo đảm đúng; mỗi mũi tên phải có điều kiện và hành động cụ thể.

2. Đếm một vật đã phát hiện không đồng nghĩa cơ cấu đã phân loại thành công vật đó.

Các bước thực hành

1. Vẽ bốn trạng thái WAIT, CLASSIFY, WAIT_CLEAR, ERROR; ghi điều kiện trên mỗi mũi tên và hành động khi vào trạng thái.

2. Chạy tay chuỗi đỏ, đỏ, trống, xanh, trống; lập bảng trạng thái trước/sau và hai bộ đếm vào/thành công.

3. Chạy chuỗi vật chưa xác định và tình huống “vật vẫn ở đó nhưng nhãn tạm mất”; không cho phép tự mở thêm chu kỳ.

4. Thêm lỗi quá thời gian gạt và phục hồi bằng kiểm tra/dừng thủ công. Kiểm thử lại toàn bộ chuỗi sau khi đổi một quy tắc chuyển.

Mã mẫu · Python · GearsBot Cage Bot (ev3dev2)

```
# GearsBot Cage Bot, ev3dev2 Python. Press Reset before Run.
from ev3dev2.motor import LargeMotor, OUTPUT_C, SpeedDPS
from ev3dev2.sensor.lego import ColorSensor, UltrasonicSensor
from time import time, sleep

try:
  color = ColorSensor("in3")
  sonar = UltrasonicSensor("in4")
  arm = LargeMotor(OUTPUT_C)
except Exception as error:
  print("SETUP:", error)
  raise RuntimeError("Select Cage Bot, then press Reset.")

def move_arm(target, deadline):
  if not 0 <= target <= 45:
    raise ValueError("Use an arm target from 0 to 45 degrees.")
  if time() >= deadline:
    return False
  limit = min(deadline, time() + 3.0)
  arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
  while abs(arm.position - target) > 2:
    if time() >= limit:
      arm.off(brake=True)
      return False
    sleep(0.02)
  arm.off(brake=True)
  return time() < limit

def read_group():
  samples = []
  for sample in range(3):
    samples.append(color.color)
    sleep(0.05)
  if samples[0] == samples[1] == samples[2]:
    if samples[0] == 5:  # EV3 red
      return "A"
    elif samples[0] == 2:  # EV3 blue
      return "B"
  return "CHECK"

state = "WAIT"
phase = ""
target = 45
phase_deadline = 0.0
clear_samples = 0
received = gate_cycles = needs_check = 0
end_time = time() + 90.0
try:
  if not move_arm(45, end_time):
    raise RuntimeError("Arm opening timeout; inspect and Reset.")
  print(state)
  while time() < end_time:
    distance = sonar.distance_centimeters
    if distance <= 0:
      state = "ERROR"
      print(state, "invalid or contact-distance reading")
      break
```

```
    if state == "WAIT" and distance <= 3.0:
      received += 1
      group = read_group()
      if group == "CHECK":
        needs_check += 1
        state = "WAIT_CLEAR"
        clear_samples = 0
        print(state, "CHECK", received)
      elif not (0 < sonar.distance_centimeters <= 3.0):
        state = "WAIT_CLEAR"
        clear_samples = 0
        print(state, "object moved")
      elif time() >= end_time:
        state = "ERROR"
        print(state, "run deadline before gate cycle")
        break
      else:
        phase, target = "CLOSE", 0
        phase_deadline = min(end_time, time() + 3.0)
        arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
        state = "CLASSIFY"
        print(state, group, "object", received)

    elif state == "CLASSIFY":
      if time() >= phase_deadline:
        state = "ERROR"
        print(state, "arm timeout", phase)
        break
      elif abs(arm.position - target) <= 2:
        arm.off(brake=True)
        if phase == "CLOSE":
          phase, target = "OPEN", 45
          phase_deadline = min(end_time, time() + 3.0)
          arm.on_to_position(SpeedDPS(30), target, brake=True, block=False)
        else:
          gate_cycles += 1
          state = "WAIT_CLEAR"
          clear_samples = 0
          print(state, "gate_cycles", gate_cycles)

    elif state == "WAIT_CLEAR":
      clear_samples = clear_samples + 1 if distance >= 8.0 else 0
      if clear_samples >= 3:
        state = "WAIT"
        print(state)
    sleep(0.05)
  if state == "CLASSIFY":
    state = "ERROR"
    print(state, "run deadline during unfinished gate cycle")
finally:
  arm.off(brake=True)
print("state", state, "received", received,
    "gate_cycles", gate_cycles, "needs_check", needs_check)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn Robot → Select Robot → Cage Bot, chuyển sang Python rồi bấm Reset trước mỗi lần chạy. Mẫu dùng ev3dev2 của GearsBot: outC là tay chắn, in3 là cảm biến màu hướng trước, in4 là siêu âm; cổng A/B dành cho hai bánh xe. Chuẩn bị vật đỏ, xanh và vàng trên sân qua Worlds → World Builder; đặt mặt màu ngang tầm cảm biến. Mở joystick để điều khiển bánh xe: tiếp cận chậm đến khoảng cách ≤ 3 cm, dừng bánh, sau chu kỳ lùi ra ≥ 8 cm. Mốc 3 cm phù hợp tầm đọc 5 cm của cảm biến màu Cage Bot. Quan sát console WAIT → CLASSIFY → WAIT_CLEAR → WAIT. Hành động hạ/mở tay chắn chỉ phát lệnh khi bắt đầu từng pha; số thành công chỉ tăng sau khi encoder xác nhận hoàn tất hai pha. Giữ vật tại chỗ hoặc đổi nhãn sang CHECK để kiểm tra chống lặp; lùi ≥ 8 cm trong ba mẫu liên tiếp mới mở lượt mới. Lỗi khoảng cách hoặc timeout chuyển sang ERROR và dừng motor; sau khi kiểm tra cơ cấu, bấm Reset để chạy lại. Số gate_cycles đo chu kỳ tay chắn; kiểm thử ngăn đích được bổ sung trên trạm phân loại thực. Đây là mã chạy cho mô hình Cage Bot; với Hub SPIKE thật, dùng API của ứng dụng LEGO Education SPIKE và cấu hình cổng của bộ lắp.

Bài tập áp dụng

Trạm đang WAIT_CLEAR sau khi gạt một vật đỏ. Các mẫu tiếp theo là đỏ, chưa biết, đỏ, trống, đỏ. Giả sử "chưa biết" chưa xác nhận vùng trống. Trạng thái nào xuất hiện và có bao nhiêu chu kỳ mới được mở? Nêu thêm phản ứng nếu chu kỳ mới bị timeout.

Gợi ý đối chiếu sau khi tự làm

Đỏ, chưa biết, đỏ đều giữ WAIT_CLEAR. Trống được xác nhận thì chuyển WAIT. Đỏ cuối tạo đúng một chu kỳ CLASSIFY mới. Nếu chu kỳ đó timeout: dừng motor, chuyển ERROR, không tăng bộ đếm thành công và không nhận vật mới cho tới khi phục hồi an toàn.

Tự kiểm tra mức hiểu

1. Sơ đồ có điều kiện chuyển và nhánh ERROR với motor dừng.

2. Hành động gạt chỉ khởi tạo một lần trong mỗi chu kỳ.

3. Bảng kiểm thử xử lý được vật đứng lâu, nhãn chưa rõ và lỗi timeout mà không đếm lặp.

Câu hỏi trắc nghiệm

Vì sao cần trạng thái chờ lấy vật ra?

1. Để tăng tốc tùy ý

2. Để tránh đếm lặp một vật

3. Để bỏ cảm biến

Đáp án: 2. WAIT_CLEAR lưu việc chu kỳ trước đã xử lý nhưng vật có thể vẫn còn trong vùng cảm biến. Chỉ cho phép nhận lại sau khi vật cũ rời đi nên tránh đếm/gạt lặp.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

LEGO Education SPIKE — ứng dụng và trợ giúp theo phiên bản

https://spike.legoeducation.com/

LEGO Education — thư viện bài học chính thức

https://education.lego.com/en-us/lessons/

GearsBot — cấu hình Cage Bot chính thức

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GearsBot — Python ev3dev2 motor API

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GearsBot — Python ev3dev2 cảm biến màu và siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py